Software Implementation and Testing Document

For

Group Hoo's Fishing

Version 1.0

Authors:
Juan Pablo Pastrana II
Tyler Dussault
Bridget Gizzi
Darius Davis
Dhruv Patel

1. Programming Languages (5 points)

GDScript continues to be the main programming language used for Hoo's Fishing and is used for the game's scripts and gameplay systems. The main reason we chose GDScript is that it is designed to work directly with Godot, making it straightforward to connect scripts with Godot features such as scenes, nodes, signals, and resources.

2. Platforms, APIs, Databases, and other technologies used (5 points)

Godot 4 is the main platform used to build the game. It is used to create the scenes, user interface, resources, and gameplay systems. The project also uses Godot resources to store game data such as fish and equipment information, but no external database is currently required because the game data is stored locally within the Godot project.

3. Execution-based Functional Testing (10 points)

We continue to perform functional testing by running the game in Godot and checking that each gameplay system works as intended before changes are merged into the main project. During Increment 2, we focused on testing newly added features such as the backpack, player tasks, catch tracking, main menu, and improvements to the fishing minigame. For example, we check that players can purchase and open the backpack, move items between the backpack and hotbar, and that the backpack cannot be opened while fishing or shopping. We also check that catching fish updates the catch tracker and progresses the appropriate player tasks. Other functional testing includes checking that rod upgrades affect the fishing minigame, the day/night cycle changes correctly, and the main menu buttons work as intended. After individual features are working, we perform integration testing to make sure the new systems work together with existing features such as fishing, inventory, the shop, and money. We also test for bugs introduced by changes, such as player movement while fishing and shop interactions during fishing.

4. Execution-based Non-Functional Testing (10 points)

We continue to perform non-functional testing by running the main game scene and using feedback from team members on the game's performance, usability, and visual appearance. During Increment 2, we focused on improving the user interface and overall gameplay experience. This included adjusting the shop and tasks interface, moving the hotbar to improve its placement, and fixing overlapping UI elements. We also made improvements to the fishing minigame and fixed visual issues such as graphical clipping and fish textures in the hotbar. Team members play the game to check that the interface is easy to use, the visuals appear correctly, and the game runs smoothly with the newly added features. Based on feedback and issues found during testing, we make changes as needed.

5. Non-Execution-based Testing (10 points)

We continue to organize non-execution-based testing through GitHub, our group chat, and team meetings. Team members review committed code and project changes, and during meetings we discuss newly added features and how they connect with existing systems. During Increment 2, we reviewed changes involving the backpack, player tasks, catch tracking, and fishing rod upgrades. For example, we improved the rod upgrade system to reduce the need to update code in multiple places when adding or changing rods. Commits and pull requests are documented so team members can understand what has been added or modified. We also review changes before and after merging them into the main project to check for conflicts and identify possible issues. This helps us maintain the project structure, improve code readability, and make sure new features work with the existing systems.